

SymPy Bug Report

1 Bug 11: `function_range` returns `EmptySet` for `sqrt(x**2)` over the reals

1.1 Minimal Reproducer

```
from sympy import S, sqrt, symbols
from sympy.calculus.util import function_range

x = symbols("x")
r = function_range(sqrt(x**2), x, S.Reals)

print("range:", r)
print("values:", sqrt((-2)**2), sqrt(0**2), sqrt(3**2))
print("contains 0:", r.contains(0))
print("contains 2:", r.contains(2))
```

1.2 Observed Behavior

```
range: EmptySet
values: 2 0 3
contains 0: False
contains 2: False
```

SymPy reports that the range is `EmptySet`. This is not merely an alternative representation of the same set: the output excludes values such as 0 and 2, even though the expression directly attains those values at $x = 0$ and $x = -2$, respectively.

1.3 Expected Behavior

The correct result is `Interval(0, ∞)`, represented in SymPy as `Interval(0, oo)`. The call

$$\text{function_range}\left(\sqrt{x^2}, x, \mathbb{R}\right)$$

should return the set of all nonnegative real numbers.

1.4 Mathematical Explanation

For real x ,

$$\sqrt{x^2} = |x|.$$

The absolute-value function is continuous on $\mathbb{R}$, has a global minimum 0 at $x = 0$, and is unbounded above. Hence

$$\{\sqrt{x^2} : x \in \mathbb{R}\} = \{|x| : x \in \mathbb{R}\} = [0, \infty).$$

Every nonnegative value y is attained, for example by taking $x = y$. The reported value $\emptyset$ is therefore a genuine loss of the whole range, not a harmless branch convention, simplification choice, or equivalent set representation.

1.5 Source-Level Diagnosis

The diagnosis locates the failure in `sympy/calculus/util.py`, inside `function_range`, especially the logic around lines 253–281 and the critical-point search at line 260. The traced call path enters `function_range`, computes `continuous_domain(sqrt(x**2), x, S.Reals)`, and obtains the full real interval. The algorithm then samples the interval endpoint limits and solves $f'(x) = 0$ with `solveset(f.diff(x), x, interval)`.

For $f(x) = \sqrt{x^2}$, both endpoint limits are ∞ . The derivative expression is `sqrt(x**2)/x`, effectively $|x|/x$ on the reals, which is undefined at the cusp $x = 0$. Consequently, the global minimum occurs at a point where the derivative is undefined rather than zero. Since `function_range` samples derivative-zero points but does not also sample interior derivative singularities where the original function remains continuous, $x = 0$ is not added as a candidate value.

With only the infinite endpoint values available, the final interval construction uses `inf = ∞` and `sup = ∞`, which collapses to `EmptySet`. The source-level behavior therefore misses the finite minimum that determines the lower endpoint of the true range. A plausible fix direction identified by the diagnosis is to include derivative singularities lying inside the original function’s continuous domain as candidate points, while intersecting such points with the original continuous domain to avoid spurious candidates.

1.6 Additional Failing Instances

The same failure appears for shifted cusp expressions of the form $\sqrt{(x+a)^2}$. Over the reals,

$$\sqrt{(x+a)^2} = |x+a|,$$

so each case has the same range $[0, \infty)$, with the minimum attained at $x = -a$. The independent verification checks the representative case and five shifted variants, comparing SymPy’s range against `Interval(0, oo)` and direct sample values computed from the real square-root formula.

Input / Expression	SymPy behavior	Expected behavior
<code>function_range(sqrt(x**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, ∞)</code>
<code>function_range(sqrt((x+1)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, ∞)</code>
<code>function_range(sqrt((x-1)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, ∞)</code>
<code>function_range(sqrt((x+2)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, ∞)</code>
<code>function_range(sqrt((x-2)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, ∞)</code>
<code>function_range(sqrt((x+3)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, ∞)</code>

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material found was documentation for `function_range`, the real-domain refinement $\sqrt{x^2} \mapsto |x|$, and the elementary-function documentation for `sqrt` and `Abs`. No public duplicate was identified for the exact `function_range(sqrt(x**2), x, S.Reals)` returning `EmptySet` symptom. The underlying failure family is, however, publicly known: the root cause is `function_range`'s critical-point search missing the non-differentiable cusp of $\sqrt{x^2} = |x|$ at $x = 0$, the same `Abs`-cusp mechanism documented in [sympy/sympy#16469](#) (`function_range(abs(x), x, S.Reals)` failing; correct only with `Symbol("x", real=True)`), [#21956](#) ("Unable to find critical points for `Abs(f(x))`"), and [#20101](#) (maximum of an `Abs` expression failing). The deduplication verdict is therefore a specific new instance of this known family rather than a wholly new failure mode with no public precedent. The case remains individually actionable because it is a small reproducible failure in `calculus.util.function_range` with a concise regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Interval, S, oo, sqrt, symbols
from sympy.calculus.util import function_range

@pytest.mark.parametrize("a", [0, 1, -1, 2, -2, 3])
def test_function_range_sqrt_shifted_square_over_reals(a):
    x = symbols("x")

    assert function_range(sqrt((x + a)**2), x, S.Reals) == Interval(0, oo)
```